Techniques for Operations on and Searching of Arrays and Lists

Computer Science 11 • BC Curriculum • 90-Minute Lesson

Lesson Roadmap

Time	Activity	Focus
0–8 min	Warm-up + review	Arrays vs. lists; indexing
8–25 min	Concept: List ops	`append`, `insert`, `remove`, `pop`
25–35 min	Visualised walkthroughs	Diagrams of each mutation
35–45 min	Think-pair-share	Choosing the right operation
45–60 min	Concept: Searching	Linear search algorithm
60–75 min	Concept: Binary search	Requirement, steps, complexity
75–83 min	Live coding practice	Guided independent work
83–90 min	Summary + exit card	Big ideas & complexity table

Learning Goals

By the end of this lesson you will be able to:

- explain the difference between a **static array** and a **dynamic list**, and identify when each is appropriate
- apply the four core list operations: `append`, `insert`, `remove`, and `pop`/delete
- implement **linear search** and explain its worst-case behaviour
- implement **binary search** and explain why it requires a sorted list
- compare the **time complexity** of linear and binary search using Big-O notation ($O(n)$ vs. $O(\log n)$)

BC CS 11 Curriculum Connection

"Techniques for operations (append, remove, insert, delete) on and searching (linear and binary searches) of arrays and lists"

Content strand: **Data structures and algorithms**

Warm-Up — What Do You Already Know? (8 min)

Quick questions

1. What is an **index**? What is the index of the first element?
2. What does `scores[2]` return from the list `scores = [88, 74, 95, 61]`?
3. How do you get the *last* element of a list without knowing its length?
4. What does `len(scores)` return?

Warm-Up — What Do You Already Know? (8 min)

Quick questions

1. What is an **index**? What is the index of the first element?
2. What does `scores[2]` return from the list `scores = [88, 74, 95, 61]`?
3. How do you get the *last* element of a list without knowing its length?
4. What does `len(scores)` return?

Answers

1. The position number; **0**
2. 95 (index 2)
3. `scores[-1]` $\Rightarrow$ 61
4. 4

Zero-based indexing is a CS convention that will feel natural with practice. Expect off-by-one errors — everyone makes them.

Arrays vs. Lists: What Is the Difference?

Static Array

- **Fixed size** set at creation
- All elements must be the **same type**
- Memory is a single contiguous block
- **Very fast** random access: $O(1)$
- Cannot grow or shrink

Used in: C, Java, low-level code

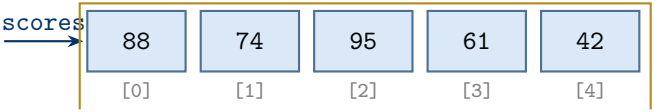
Dynamic List (Python)

- **Variable size** — grows and shrinks
- Can hold **mixed types**
- Internally uses a resizable array
- Still $O(1)$ random access
- Operations like append are $O(1)$ amortised

Python's `list` is a dynamic array.

In Python we use lists. The word "array" in the BC CS 11 curriculum refers to the underlying concept. Everything we do today uses Python lists, which behave as dynamic arrays.

How a List Lives in Memory



contiguous block in memory

Random access is $O(1)$
To get element i: start address $+i\times$ element size. No looping needed — one calculation.

Insertion and deletion are $O(n)$
Inserting in the middle forces every subsequent element to shift one position. Up to n moves in the worst case.

The Four Core List Operations

Operation	Python method	Where?	Complexity
Append	`list.append(x)`	End of list	$O(1)$ amortised
Insert	`list.insert(i, x)`	Any index i	$O(n)$
Remove	`list.remove(x)`	First match by value	$O(n)$
Delete	`list.pop(i)`	By index (default: last)	$O(1)$ end / $O(n)$ mid

Why does position matter for complexity? `append` adds to the end — no shifting required. `insert(0, x)` inserts at the front — *every* existing element must move one step right. With n elements that is n shifts.

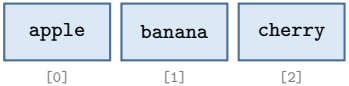
Operation 1 — append

Adds one element to the <u>end</u> of the list.

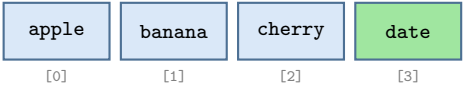
```python
fruits = ["apple", "banana", "cherry"]
fruits.append("date")
print(fruits)
# ["apple", "banana", "cherry", "date"]
```

```python
# Building a list from user input
scores = []
for i in range(5):
    val = int(input("Score: "))
    scores.append(val)
print("Collected:", scores)
```

Before:



After `append("date")`**:**



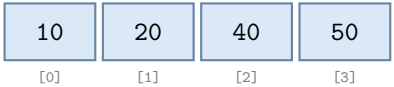
Operation 2 — insert

Inserts an element <u>before</u> the given index. Existing elements shift right.

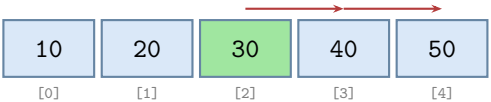
```python
nums = [10, 20, 40, 50]
nums.insert(2, 30)    # insert 30 at index 2
print(nums)
# [10, 20, 30, 40, 50]
```

```python
# Insert in sorted order (manual)
data = [1, 3, 5, 7]
data.insert(2, 4)    # [1, 3, 4, 5, 7]
data.insert(0, 0)    # [0, 1, 3, 4, 5, 7]
```

Before `insert(2, 30)`:



After — elements at ≥ 2 shift:

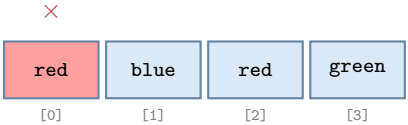


Operation 3 — remove

Removes the first occurrence of a value. Raises `ValueError` if not found.

```python
colours = ["red","blue","red","green"]
colours.remove("red")
print(colours)
# ["blue", "red", "green"]

# Always check first to avoid crash
target = "purple"
if target in colours:
    colours.remove(target)
else:
    print(target, "not found")
```

Before `remove("red")`:



After — remaining elements shift left:

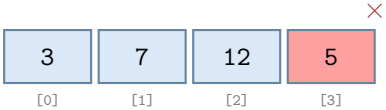
Two costs: (1) linear scan to find the first match — $O(n)$; (2) left-shift of all subsequent elements

Operation 4 — pop / Delete by Index

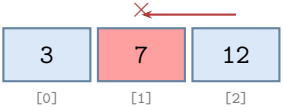
Removes and <u>returns</u> the element at a given index. Default: last element.

```python
stack = [3, 7, 12, 5]

# Remove last (fast: O(1))
last = stack.pop()
print(last)    # 5
print(stack)   # [3, 7, 12]

# Remove by index (O(n) if not last)
mid = stack.pop(1)
print(mid)     # 7
print(stack)   # [3, 12]

# del keyword (no return value)
data = [10, 20, 30, 40]
del data[2]
```

`pop()` — **remove last ($O(1)$):**



`pop(1)` — **remove middle ($O(n)$):**



`pop()` (no argument) is the go-to for **stack**

Operations Summary and When to Use Each

Method	What it does	Use when...	Cost
`append(x)`	Adds `x` to the end	Building a list; adding items one by one	$O(1)$
`insert(i, x)`	Inserts `x` before index `i`	Maintaining sorted order; inserting mid-list	$O(n)$
`remove(x)`	Removes first `x` by value	You know the value, not the position	$O(n)$
`pop(i)`	Removes & returns element at `i`	You know the index; stack-/queue use	$O(1)$ end $O(n)$ mid
`del lst[i]`	Removes element at `i` (no return)	Discarding by index, no return value needed	$O(n)$ mid

Think-Pair-Share — Choose the Right Operation (10 min)

Scenario

You have a list of student names sorted alphabetically:

```
roster = ["Alice","Charlie","Eve","Grace","Mallory"]
```

For each task, **name the operation, write the call, and state why**:

1. A new student "Frank" enrols. Maintain alphabetical order.
2. "Eve" withdraws. You know her name but not her index.
3. "Grace" moves to a different class. You know she is at index 3.
4. At the end of term, add "Zara" to the list.

Think-Pair-Share — Choose the Right Operation (10 min)

Scenario

You have a list of student names sorted alphabetically:

```
roster = ["Alice","Charlie","Eve","Grace","Mallory"]
```

For each task, **name the operation, write the call, and state why**:

1. A new student "Frank" enrols. Maintain alphabetical order.
2. "Eve" withdraws. You know her name but not her index.
3. "Grace" moves to a different class. You know she is at index 3.
4. At the end of term, add "Zara" to the list.

Answers: (1) `roster.insert(3,"Frank")` — alphabetical position; (2) `roster.remove("Eve")` — value known; (3) `roster.pop(3)` — index known, efficient; (4) `roster.append("Zara")` — end of list, $O(1)$.

Searching: The Core Problem

Given a list and a target, find the target's index (or confirm it is absent).

Two classic algorithms:

- **Linear Search** — check every element one by one. *Works on any list.*
- **Binary Search** — divide and conquer. *Requires a sorted list.*

Real-world analogies:

- Linear: flipping through a deck of cards one by one
- Binary: opening a dictionary at the middle, then the relevant half

Why does algorithm choice matter?

Searching a list of 1 000 000 elements:

- Linear worst case: **1 000 000 comparisons**
- Binary worst case: **20 comparisons** ($\lceil \log_2 1\,000\,000 \rceil = 20$)

Algorithm choice is the biggest lever for performance.

Linear Search — The Algorithm

Algorithm (pseudocode)

Input: list `data`, target value `key`

Output: index of `key`, or -1 if not found

1. Start at index $i = 0$
2. **While** $i <$ `len(data)`:
   1. If `data[i]` $=$ `key`: **return** i
   2. $i \leftarrow i + 1$
3. **Return** -1

Best case: target is at index 0 — **1 comparison**, $O(1)$

Average case: target is in the middle — $O(n/2) \approx O(n)$

Advantages:

- Works on *any* list (unsorted OK)
- Simple to code and understand
- No extra memory required

Linear Search — Visual Walkthrough

Search for `61` **in** `[88, 74, 95, 61, 42]`

	[0]	[1]	[2]	[3]	[4]	
Step 1 (i=0)	88	74	95	61	42	$88 \neq 61$, continue
Step 2 (i=1)	88	74	95	61	42	$74 \neq 61$, continue
Step 3 (i=2)	88	74	95	61	42	$95 \neq 61$, continue
Step 4 (i=3)	88	74	95	61	42	$61 = 61$ — **FOUND at index 3!**

Linear Search — Python Implementation

```python
def linear_search(data, key):
    """Return index of key or -1."""
    for i in range(len(data)):
        if data[i] == key:
            return i    # found!
    return -1           # not found

# --- Example usage ---
scores = [88, 74, 95, 61, 42]

idx = linear_search(scores, 61)
if idx != -1:
    print(f"Found 61 at index {idx}")
else:
```

Tracing execution:

i=0: data[0]=88, $88 \neq 61$
i=1: data[1]=74, $74 \neq 61$
i=2: data[2]=95, $95 \neq 61$
i=3: data[3]=61, $61 = 61$ ✓
Returns 3

Python shortcut:

"in" uses linear search internally:

`if 61 in scores:`

Writing it explicitly teaches the algorithm.

Binary Search — The Big Idea

Critical Prerequisite

Binary search **only works on a sorted list**. Always verify this before applying binary search.

Core strategy — Divide and Conquer:

1. Look at the **middle** element
2. If it equals the target: **done**
3. If target $<$ middle: search the **left half**
4. If target $>$ middle: search the **right half**
5. Repeat until found or the search space is empty

Why is this so fast?

Each step *halves* the remaining candidates.

Start with n elements.
After 1 step: $n/2$ remain.
After 2 steps: $n/4$ remain.
After k steps: $n/2^k$ remain.

Done when $n/2^k = 1$, i.e. $k = \log_2 n$.

Worst case: $O(\log n)$

Binary Search — Visual Walkthrough

Search for `37` **in** `[5, 12, 20, 31, 37, 44, 58, 72, 91]`

	lo=0				mid=4				hi=8	
Step 1	5	12	20	31	37	44	58	72	91	`data[4]=37=37` ⇒ **FOUND**
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	

Alternative: searching for 20 (shows 3 steps)

Step 1	5	12	20	31	37	44	58	72	91	`data[4]=37>20` ⇒ search lef
Step 2	5	12	20	31	37	44	58	72	91	`data[1]=12<20` ⇒ search rig

Binary Search — Python Implementation

```python
def binary_search(data, key):
    """Sorted list required.
       Returns index or -1."""
    low = 0
    high = len(data) - 1

    while low <= high:
        mid = (low + high) // 2

        if data[mid] == key:
            return mid          # found
        elif data[mid] < key:
            low = mid + 1       # right half
        else:
            high = mid - 1      #
```

Tracing `key=37`:

`low=0, high=8`
`mid=4, data[4]=37=37`
Return 4

Tracing `key=20`:

`low=0, high=8, mid=4`
`data[4]=37 > 20`
`high = 3`
`low=0, high=3, mid=1`

Why Must the List Be Sorted?

Counter-example

```
data = [5, 91, 20, 37, 12]  # unsorted!
# Searching for 37:
# mid=2, data[2]=20 < 37  -> go right
# low=3, high=4, mid=3
# data[3]=37  -> found at 3
# Seems to work... but:

data2 = [5, 91, 37, 20, 12]
# Searching for 37:
# mid=2, data[2]=37  -> found at 2  OK
# Searching for 20:
# mid=2, data[2]=37 > 20 -> go left
```

The key insight:

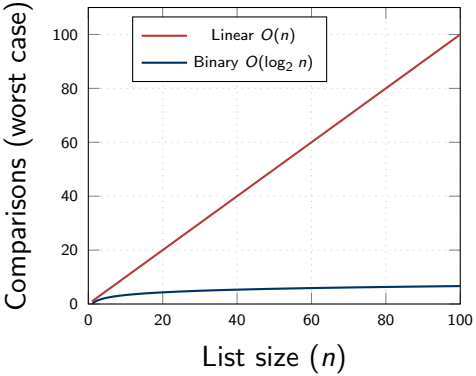
Binary search relies on the guarantee that *everything to the left of mid is smaller* and *everything to the right is larger*.

Without sorted order, discarding half the list is **unjustified** — the target could be in the discarded half.

Always sort first (or maintain sorted order on insert) if you plan to use binary search repeatedly.

Python's built-in:

Time Complexity: Linear vs. Binary Search



n	Linear	Binary
10	10	4
100	100	7
1 000	1000	10
10 000	10000	14
1 000 000	1 000 000	20

As n grows, binary search becomes overwhelmingly faster. At n=1,000,000: linear checks 1 000 000 elements; binary checks just **20**.

Choosing the Right Search Algorithm

Situation	Use	Reason
List is **unsorted**	Linear	Binary requires sorted order
List is sorted, searched **once**	Either	Overhead of sorting may not pay off
List is sorted, searched **many times**	Binary	$O(\log n)$ pays off quickly
List is **small** ($n < 20$)	Linear	Simpler code; difference negligible
List is **large** ($n > 1000$)	Binary	Dramatic speed advantage
Need **simplest** code	Linear	One loop, easy to read and debug

Guided Practice (8 min) — Problems

Complete in Python. Show all code and trace for Question 3.

1. Start with `temps = [18, 22, 15, 30, 27]`. Add 25 to the end. Insert 10 at the front. Remove the value 15. Print the final list.
2. Write a function `count_occurrences(data, key)` that uses linear search to count *how many times* `key` appears in `data` (do not use `data.count()`).
3. Trace binary search on `data = [3, 9, 14, 21, 35, 47, 52]` searching for 14. Show `low`, `high`, and `mid` at each step.
4. *Challenge:* Modify `binary_search` so that instead of returning -1 when the key is absent, it returns the **index where the key should be inserted** to keep the list sorted.

Practice Solutions

Q1:

```
temps = [18, 22, 15, 30, 27]
temps.append(25)
temps.insert(0, 10)
temps.remove(15)
print(temps)  # [10, 18, 22,
    30, 27, 25]
```

Q2:

```
def count_occurrences(data,
    key):
    count = 0
    for item in data:
        if item == key:
            count += 1
    return count
```

Q3 trace (searching 14 in [3,9,14,21,35,47,52]):

Step	low	high	mid
1	0	6	3 (data[3]=21)
	$21 > 14 \Rightarrow$ high=2		
2	0	2	1 (data[1]=9)
	$9 < 14 \Rightarrow$ low=2		
3	2	2	2 (data[2]=14)
	Found at index 2!		

Lesson Summary — The Big Ideas

List Operations

- `append(x)` — add to end, $O(1)$
- `insert(i, x)` — add anywhere, $O(n)$
- `remove(x)` — by value, $O(n)$
- `pop(i)` — by index; $O(1)$ at end, $O(n)$ mid

Inserting/removing in the **middle** shifts elements — that is why these are $O(n)$.

Searching

- **Linear:** check each element; $O(n)$; works unsorted
- **Binary:** halve the search space each step; $O(\log n)$; *sorted list required*

With n=1,000,000: linear = 10^6 steps; binary = 20 steps. Algorithm choice is performance.

BC CS 11 curriculum connections: *Decomposition* — both search algorithms break the problem into smaller sub-problems. *Algorithms are essential* — we chose between two correct algorithms based on what makes one *better* than the other.

Exit Card

Answer before you leave — on paper or online

(a) What is the output of the following code?

```
x = [4, 8, 2, 5, 9]
x.insert(2, 6)
x.remove(8)
x.append(1)
print(x)
```

(b) A sorted list has 512 elements. What is the *maximum* number of comparisons binary search makes?

(c) Explain in one sentence why binary search cannot be used on `[5, 3, 8, 1, 7]`.

(d) A classmate says: *"`remove` is always $O(1)$ because Python is fast."* Write a one-sentence correction.

Exit Card

Answer before you leave — on paper or online

(a) What is the output of the following code?

```
x = [4, 8, 2, 5, 9]
x.insert(2, 6)
x.remove(8)
x.append(1)
print(x)
```

(b) A sorted list has 512 elements. What is the *maximum* number of comparisons binary search makes?

(c) Explain in one sentence why binary search cannot be used on `[5, 3, 8, 1, 7]`.

(d) A classmate says: *"`remove` is always $O(1)$ because Python is fast."* Write a one-sentence correction.

Answers:
(a) `[4, 6, 2, 5, 9, 1]`
(b) $\lceil \log_2 512 \rceil = \mathbf{9}$ comparisons
(c) The list is unsorted; binary search needs guaranteed ordering to safely discard half the search space.